# Занятие 10: Nginx и Reverse Proxy

## Веб-сервер, раздача статики, HTTPS, безопасность

## Слайд 1: Что такое веб-сервер?

**Веб-сервер - программа, принимающая HTTP-запросы**

**Основные функции:**

- Приём HTTP-запросов
- Раздача статических файлов
- Передача запросов приложению (reverse proxy)
- Балансировка нагрузки
- SSL/TLS терминирование
- Логирование

**Популярные веб-серверы:**

- **Nginx** - быстрый, event-driven
- **Apache** - гибкий, модульный
- **Caddy** - автоматический HTTPS

## Слайд 2: Nginx - что это?

**Nginx (engine-x)** - высокопроизводительный веб-сервер

**Особенности:**

- Event-driven архитектура (асинхронный)
- Низкое потребление памяти
- Высокая производительность
- Reverse proxy
- Load balancing
- HTTP/2 поддержка

**Архитектура:**

```
Master process
  ├─ Worker process 1
  ├─ Worker process 2
  └─ Worker process N
```

Каждый worker обрабатывает тысячи соединений асинхронно.

## Слайд 3: Базовая конфигурация Nginx

### Пример конфигурации:

```
server {
    listen 80;
    server_name example.com www.example.com;

    root /var/www/html;
    index index.html;

    location / {
        try_files $uri $uri/ =404;
    }

    location /images/ {
        alias /var/www/images/;
    }
}
```

### Директивы:

- `listen` - порт для прослушивания
- `server_name` - доменное имя
- `root` - корневая директория
- `location` - правила для путей

## Слайд 4: Раздача статических файлов

### Пример конфигурации для статики:

```
server {
    listen 80;

    # CSS и JS
    location /static/ {
        alias /var/www/static/;
        expires 30d;
        add_header Cache-Control "public, immutable";
    }

    # Изображения
    location /images/ {
        alias /var/www/images/;
        expires 1y;
        add_header Cache-Control "public, immutable";
    }

    # Сжатие
    gzip on;
    gzip_types text/css application/javascript image/svg+xml;
}
```

### Оптимизации:

- `expires` - кэширование в браузере
- `gzip` - сжатие
- `Cache-Control` - управление кэшем

## Слайд 5: Reverse Proxy

### Что такое reverse proxy?

Сервер, который принимает запросы и передаёт их другому серверу.

**Схема:**

```
Client → Nginx (reverse proxy) → Application
```

**Зачем нужен:**

- Скрытие внутреннего приложения
- SSL/TLS терминирование
- Балансировка нагрузки
- Кэширование
- Безопасность

## Слайд 6: Конфигурация Reverse Proxy

### Пример:

```
server {
    listen 80;
    server_name api.example.com;

    location / {
        proxy_pass http://localhost:8000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

### proxy_set_header:

- `Host` - передаёт оригинальный Host
- `X-Real-IP` - реальный IP клиента
- `X-Forwarded-For` - цепочка прокси
- `X-Forwarded-Proto` - протокол (http/https)

## Слайд 7: Upstream и балансировка

### Upstream - группа серверов:

```
upstream backend {
    server localhost:8000;
    server localhost:8001;
    server localhost:8002;
}

server {
    listen 80;

    location / {
        proxy_pass http://backend;
    }
}
```

### Методы балансировки:

- `round-robin` (по умолчанию) - по очереди
- `least_conn` - меньше всего соединений
- `ip_hash` - по IP клиента

```
upstream backend {
    least_conn;
    server localhost:8000;
    server localhost:8001;
}
```

## Слайд 8: HTTPS и SSL

### Зачем HTTPS:

- Шифрование данных
- Защита от перехвата
- Доверие пользователей
- SEO преимущество

### Получение сертификата:

- Let's Encrypt (бесплатно)
- Коммерческие сертификаты

### Установка сертификата:

```
server {
    listen 443 ssl http2;
    server_name example.com;

    ssl_certificate /etc/ssl/certs/example.com.crt;
    ssl_certificate_key /etc/ssl/private/example.com.key;

    location / {
        proxy_pass http://localhost:8000;
    }
}
```

## Слайд 9: Let's Encrypt и Certbot

### Автоматическое получение сертификата:

```
# Установка certbot
sudo apt install certbot python3-certbot-nginx

# Получение сертификата
sudo certbot --nginx -d example.com -d www.example.com

# Автоматическое обновление
sudo certbot renew --dry-run
```

### Nginx конфигурация (автоматически):

```
server {
    listen 443 ssl;
    server_name example.com;

    ssl_certificate /etc/letsencrypt/live/example.com/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/example.com/privkey.pem;

    include /etc/letsencrypt/options-ssl-nginx.conf;
}
```

## Слайд 10: HTTP → HTTPS redirect

### Принудительное HTTPS:

```
server {
    listen 80;
    server_name example.com;

    return 301 https://$server_name$request_uri;
}

server {
    listen 443 ssl http2;
    server_name example.com;

    ssl_certificate /etc/letsencrypt/live/example.com/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/example.com/privkey.pem;

    location / {
        proxy_pass http://localhost:8000;
    }
}
```

### HSTS (HTTP Strict Transport Security):

```
add_header Strict-Transport-Security "max-age=31536000; includeSubDomains" always;
```

## Слайд 11: Тайм-ауты

### Настройка тайм-аутов:

```
server {
    location / {
        proxy_pass http://localhost:8000;

        # Тайм-ауты
        proxy_connect_timeout 60s;
        proxy_send_timeout 60s;
        proxy_read_timeout 60s;
    }

    # Для долгих запросов
    location /api/long-running {
        proxy_pass http://localhost:8000;
        proxy_read_timeout 300s;
    }
}
```

### Типы тайм-аутов:

- `proxy_connect_timeout` - соединение с upstream
- `proxy_send_timeout` - отправка запроса
- `proxy_read_timeout` - чтение ответа

## Слайд 12: Логирование

### Access log и error log:

```
http {
    log_format main '$remote_addr - $remote_user [$time_local] "$request" '
                    '$status $body_bytes_sent "$http_referer" '
                    '"$http_user_agent" "$http_x_forwarded_for"';

    access_log /var/log/nginx/access.log main;
    error_log /var/log/nginx/error.log warn;
}

server {
    access_log /var/log/nginx/api_access.log main;
    error_log /var/log/nginx/api_error.log;
}
```

### Формат лога:

- `$remote_addr` - IP клиента
- `$request` - HTTP запрос
- `$status` - статус-код
- `$request_time` - время выполнения

## Слайд 13: Коды ошибок

### Распространённые коды:

| Код | Значение | Причина |
|---|---|---|
| 404 | Not Found | Файл не найден |
| 500 | Internal Server Error | Ошибка приложения |
| 502 | Bad Gateway | Upstream недоступен |
| 504 | Gateway Timeout | Upstream слишком долго отвечает |

### Кастомные страницы ошибок:

```
error_page 404 /404.html;
error_page 500 502 503 504 /50x.html;

location = /50x.html {
    root /usr/share/nginx/html;
}
```

## Слайд 14: Безопасность Nginx

### Заголовки безопасности:

```
server {
    # Скрыть версию Nginx
    server_tokens off;

    # Защитные заголовки
    add_header X-Frame-Options "SAMEORIGIN" always;
    add_header X-Content-Type-Options "nosniff" always;
    add_header X-XSS-Protection "1; mode=block" always;
    add_header Referrer-Policy "no-referrer-when-downgrade" always;

    # Ограничение размера запроса
    client_max_body_size 10M;
}
```

### Ограничения:

- `client_max_body_size` - максимальный размер тела
- `limit_req` - ограничение частоты запросов

## Слайд 15: Ограничение частоты запросов

### Rate limiting:

```
http {
    # Зона для rate limiting
    limit_req_zone $binary_remote_addr zone=api:10m rate=10r/s;

    server {
        location /api/ {
            limit_req zone=api burst=20 nodelay;
            proxy_pass http://localhost:8000;
        }
    }
}
```

### Параметры:

- `zone=api:10m` - зона 10MB
- `rate=10r/s` - 10 запросов в секунду
- `burst=20` - burst до 20 запросов
- `nodelay` - немедленная обработка burst

## Слайд 16: Docker Compose с Nginx

### Пример docker-compose.yml:

```yaml
version: '3.8'

services:
  app:
    build: .
    ports:
      - "8000:8000"

  nginx:
    image: nginx:alpine
    ports:
      - "80:80"
      - "443:443"
    volumes:
      - ./nginx.conf:/etc/nginx/nginx.conf
      - ./static:/var/www/static
    depends_on:
      - app
```

### nginx.conf:

```
server {
    listen 80;

    location / {
        proxy_pass http://app:8000;
    }

    location /static/ {
        alias /var/www/static/;
```

## Слайд 17: Практическое задание

### Задание 1: Базовая конфигурация

- Установить Nginx
- Создать простую страницу
- Настроить раздачу статики

### Задание 2: Reverse Proxy

- Настроить proxy_pass к приложению
- Добавить proxy_set_header
- Проверить X-Forwarded-For

### Задание 3: HTTPS

- Получить сертификат Let's Encrypt
- Настроить HTTPS
- Настроить redirect HTTP → HTTPS

### Задание 4: Балансировка

- Настроить upstream с 2 серверами
- Проверить балансировку
- Настроить least_conn

### Задание 5: Безопасность

## Слайд 18: Вопрос для собеседования

**Вопрос:** Проследите запрос через веб-сервер и выберите место проверки при сбое.

**Ожидаемый ответ:**

1. Клиент отправляет запрос на Nginx
2. Nginx проверяет конфигурацию (location, proxy_pass)
3. Nginx передаёт запрос приложению (upstream)
4. Приложение обрабатывает запрос
5. При сбое:
    - 404 - проверить location и файлы
    - 502 - проверить upstream (приложение запущено?)
    - 504 - проверить тайм-ауты (приложение отвечает?)
    - 500 - проверить логи приложения